Daemon

Indice dei contenuti

- 1. Testo
  - 1.1. Script Python
  - 1.2. Servizio
- 2. Suggerimenti
  - 2.1. Manuali di riferimento del sistema
  - 2.2. Script Python
    - 2.2.1. Esecuzione del programma
    - 2.2.2. Modalità interattiva
    - 2.2.3. Documentazione
    - 2.2.4. Struttura del programma
    - 2.2.5. Ricorsione
  - 2.3. Servizio
    - 2.3.1. Struttura del file unit
    - 2.3.2. Posizione del file unit
    - 2.3.3. Gestione del servizio
    - 2.3.4. Log
- 3. Soluzione
- Licenze

1. Testo

1.1. Script Python

Scrivi uno script Python che analizzi periodicamente una directory specificata (incluse tutte le sue sottodirectory), identificando i file la cui dimensione supera o è uguale a una certa soglia (in byte). Ogni volta che lo script trova un file la cui dimensione è maggiore o uguale alla soglia, deve scrivere il percorso del file in un file di log. Nella tua home directory, crea una directory chiamata `large-file-detector`, e al suo interno il file `app.py`, usando questo template:

```python
# nome e cognome:
# matricola:
#
# percorso:

import argparse
import os
import sys
import time

def main():
    pass


if __name__ == "__main__":
    main()
```

Lo script deve accettare esattamente quattro argomenti obbligatori da riga di comando, analizzati con il modulo `argparse` : `--target` , che indica il percorso assoluto della directory da controllare; `--size` , che specifica la dimensione minima in byte (intero positivo) dei file da segnalare; `--interval` , che definisce l'intervallo in secondi (intero positivo) tra ogni controllo; e infine `--log` , che specifica dove salvare il file di log. Il nome del file di log è sempre `large-file-detector.log` .

Dopo il parsing, valida gli input ricevuti: verifica che il percorso specificato da `--target` sia assoluto (`os.path.isabs`), esista (`os.path.exists`) e sia una directory (`os.path.isdir`); verifica anche che i valori forniti per `--size` e `--interval` siano interi positivi; verifica che il percorso indicato da `--log` esista (`os.path.exists`) e sia una directory (`os.path.isdir`).

Dopo la validazione, attraversa ricorsivamente l'albero di directory al percorso fornito con `--target` (`os.listdir` , `os.path.join` , `os.path.isdir`). Per ogni file (`os.path.isfile`), calcola la sua dimensione (`os.path.getsize`) e confrontala con la soglia definita da `--size` . Se la dimensione è maggiore o uguale alla soglia indicata, scrivi il percorso assoluto del file, seguito da un carattere di nuova riga, nel file di log chiamato `large-file-detector.log` (`open`), situato nella directory specificata con `--log` . Ogni operazione di scrittura nel file di log deve essere in modalità append. Lo script deve ripetere questa procedura periodicamente, attendendo un numero di secondi pari al valore indicato da `--interval` tra ogni controllo (`time.sleep`).

Ad esempio, eseguendo

```
$ python ~/large-file-detector/app.py \
    --target ~/archive \
    --size 10 \
    --interval 30 \
    --log ~
```

lo script identificherà tutti i file di `10` byte o più in `~/archive` (e tutte le sue sottodirectory) e aggiungerà il percorso di ogni file identificato a `~/large-file-detector.log` . Lo script ripeterà questa operazione ogni `30` secondi.

1.2. Servizio

Crea un'unità di servizio chiamata `large-file-detector.service` nell'istanza `systemd` del tuo utente. L'unità deve avviare `~/large-file-detector/app.py` con gli argomenti `--target %h/docs` , `--size 100` , `--interval 300` e `--log %h` , avviarsi al boot del sistema e riavviarsi in caso di guasti. Usa questo template:

```
# nome e cognome:
# matricola:
#
# percorso:
#
# comando per abilitare il servizio:
# comando per avviare il servizio:
```

2. Suggerimenti

2.1. Manuali di riferimento del sistema

`man` visualizza i manuali di riferimento del sistema. Ad esempio, `man:systemctl(1)` si traduce in

```
$ man 1 systemctl
```

2.2. Script Python

2.2.1. Esecuzione del programma

Per eseguire un programma Python

```
$ python3 <tuo_programma>.py
```

> [!warning]
> A seconda del tuo sistema, potrebbe essere necessario usare `python` o `python3`.

2.2.2. Modalità interattiva

La modalità interattiva avvia un ciclo read-eval-print (REPL) in cui puoi digitare espressioni Python e vederne immediatamente i risultati. Questo è utile per sperimentare con funzioni e moduli prima di scrivere un programma completo.

Per abilitare la modalità interattiva

```
$ python3
```

2.2.3. Documentazione

`help()` visualizza la documentazione per moduli, funzioni, classi e parole chiave nell'interprete interattivo Python. Ad esempio, per leggere la documentazione del modulo `argparse`

```
$ python3
>>> import argparse
>>> help(argparse)
```

Per ottenere aiuto su una funzione specifica

```
$ python3
>>> import os.path
>>> help(os.path.isabs)
```

2.2.4. Struttura del programma

Il seguente estende il template da §1.1 con commenti che corrispondono a ogni passaggio descritto nel testo.

```python
# nome e cognome:
# matricola:
#
# percorso:

import argparse
import os
import sys
import time


def walk(basepath, size, log_path):
    # attraversamento ricorsivo
    pass


def main():
    # parsing degli argomenti

    # validazione

    # esecuzione periodica
    pass


if __name__ == "__main__":
    main()
```

2.2.5. Ricorsione

Il seguente esempio elenca ricorsivamente tutti i percorsi dei file in un albero di directory.

```python
import os


def walk(basepath):
    for filename in os.listdir(basepath):
        path = os.path.join(basepath, filename)
        if os.path.isfile(path):
            print(path)
        elif os.path.isdir(path):
            walk(path)
```

2.3. Servizio

2.3.1. Struttura del file unit

Il seguente estende il template da §1.2 con commenti che corrispondono a ogni sezione del file unit.

```
# nome e cognome:
# matricola:
#
# percorso:
#
# comando per abilitare il servizio:
# comando per avviare il servizio:

[Unit]
# descrizione leggibile

[Service]
# come eseguire il servizio
# cosa fare in caso di guasto

[Install]
# avvio al boot
```

Un esempio di file unit di servizio è disponibile qui. Le direttive disponibili sono documentate in `man:systemd.unit(5)` (per `[Unit]` e `[Install]`) e `man:systemd.service(5)` (per `[Service]`).

> [!tip]
> `%h` si espande nella home directory dell'utente che esegue il servizio. Ad esempio, `WorkingDirectory=%h/large-file-detector` si espande in `WorkingDirectory=/home/ubuntu/large-file-detector` .

> [!tip]
> Aggiungi `Environment=PYTHONUNBUFFERED=1` a `[Service]` in modo che l'output di `print` appaia immediatamente in `journalctl` .

2.3.2. Posizione del file unit

Le directory in cui `systemd` legge i file unit sono documentate in `man:systemd.unit(5)` . La directory consigliata per le unità utente è `~/.config/systemd/user` . Se non esiste, creala

```
$ mkdir -p ~/.config/systemd/user
```

Poi copia il file unit lì

```
$ cp ~/large-file-detector/large-file-detector.service ~/.config/systemd/user
```

2.3.3. Gestione del servizio

I comandi per controllare `systemd` sono elencati qui. Aggiungi l'opzione `--user` per controllare l'istanza utente invece di quella di sistema. Vedi anche `man:systemctl(1)` .

Dopo aver copiato il file unit:

- ☐ Elenca i file unit per verificare che `systemd` veda il nuovo file (`list-unit-files`)
- ☐ Ricarica i file unit in modo che `systemd` recepisca le modifiche (`daemon-reload`)
- ☐ Avvia il servizio (`start`)
- ☐ Controlla lo stato per confermare che sia stato caricato correttamente e sia in esecuzione (`status`)
- ☐ Abilita il servizio in modo che si avvii al boot (`enable`)
- ☐ Riavvia il sistema
- ☐ Controlla lo stato per confermare che sia sopravvissuto al riavvio (`status`)
- ☐ Termina il servizio per simulare un guasto (`kill`)
- ☐ Controlla lo stato per confermare che `systemd` lo abbia riavviato (`status`)

> [!tip]
> Usa `systemctl --user kill --signal=SIGKILL large-file-detector.service` per terminare forzatamente il tuo servizio e verificare che `systemd` lo riavvii.

2.3.4. Log

`journalctl` interroga il journal di `systemd` . Aggiungi l'opzione `--user` per leggere i log dall'istanza utente. Ad esempio

```
$ journalctl --user -u large-file-detector.service
```

Vedi anche `man:journalctl(1)` .

3. Soluzione

Questo esercizio è stato proposto il 20 giugno 2025.

Licenze

Contenuto	Licenza
Codice	MIT License
Testo	Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International